

Day 1 — Neural Networks: Interview Revision

Topics: Neural network architecture, neurons, weights, biases, forward propagation, activation functions, Universal Approximation Theorem, output activations, parameter counting, and common interview traps.

How to revise: Read the question first, answer it aloud from memory, then check the concise answer. Focus especially on the boxed interview answers and the final rapid-fire section.

1. Neural Network Basics

Q1. What is a neural network?

A neural network is a model composed of layers of interconnected neurons. Each neuron computes a weighted combination of its inputs plus a bias, then typically applies an activation function. The output of one layer becomes the input to the next layer.

Core equation:

$$z = Wx + b$$

$$a = f(z)$$

Q2. What does a single neuron compute?

A neuron first computes a weighted sum of its inputs plus a bias, then applies an activation function.

Equation:

$$z = w^T x + b$$

$$a = f(z)$$

For three features: $z = w_1x_1 + w_2x_2 + w_3x_3 + b$.

Q3. Why do we need a bias?

Without a bias, a simple linear model is forced through the origin. The bias shifts the function/decision boundary, giving the model more flexibility.

$$y = wx + b$$

w controls the slope; b controls the intercept.

Q4. Why do we need activation functions?

Without nonlinear activation functions, stacking multiple linear layers is still equivalent to one linear/affine transformation. Nonlinear activations allow neural networks to represent complex nonlinear functions and decision boundaries.

2. Linear Layers and Nonlinearity

Q5. What happens if all layers are linear?

All linear/affine layers can be collapsed into a single affine transformation. Therefore, adding more linear layers does not increase the class of functions the network can represent.

If $h = W_1x + b_1$ and $y = W_2h + b_2$, then y can be rewritten as $W'x + b'$.

Q6. Why can a neural network learn nonlinear decision boundaries?

Because nonlinear activation functions are inserted between linear transformations. A network with only linear layers can learn only linear boundaries; adding nonlinear activations allows complex nonlinear boundaries.

Q7. What if the first hidden layer has no activation, but later layers do?

The network can still learn nonlinear functions. The first layer is simply an affine transformation, while a later nonlinear activation introduces nonlinearity into the overall composition.

Q8. Why is a network with no hidden layers and a softmax output limited?

It is essentially multinomial logistic regression. The logits are linear functions of the inputs, so it can learn only linear decision boundaries. The number of output neurons is determined by the number of classes, not the number of input features.

3. Universal Approximation Theorem and Depth

Q9. Why are neural networks universal approximators?

The Universal Approximation Theorem states, under appropriate conditions, that a feed-forward neural network with at least one hidden layer and a suitable nonlinear activation can approximate a broad class of continuous functions on a compact domain to arbitrary accuracy, given sufficient hidden units.

Important: It is about representational capacity. It does not guarantee successful optimization, a practical number of neurons, or good generalization.

Q10. Why not use one enormous hidden layer for every problem?

A sufficiently wide shallow network can approximate many functions, but it may require an impractically large number of neurons. Deep networks exploit hierarchical/compositional structure and can represent many complex functions more efficiently.

Example: pixels → edges → textures → parts → objects.

Q11. What does the Universal Approximation Theorem NOT guarantee?

It does not guarantee that the network will learn the correct parameters, that training will converge to the desired solution, that the model will generalize well, or that the required network will be small or computationally practical.

4. ReLU and Activation Functions

Q12. What is ReLU?

ReLU (Rectified Linear Unit) is a nonlinear, piecewise-linear activation function.

$$\text{ReLU}(x) = \max(0, x)$$

Its derivative is 1 for $x > 0$ and 0 for $x < 0$. It divides the input into active and inactive regions.

Q13. Why is ReLU commonly used instead of sigmoid/tanh in hidden layers?

- Better gradient flow:** ReLU does not saturate on the positive side; its derivative is 1 for positive inputs, reducing saturation-related vanishing gradients.
- Computational efficiency:** $\max(0, x)$ is cheaper to compute than exponential-based sigmoid/tanh functions.
- Sparse activations:** negative inputs produce exact zeros, so only some neurons may be active for a given input.

Q14. Is ReLU a linear activation?

No. ReLU is **nonlinear** but **piecewise linear**. This distinction is important in interviews.

Q15. What is the dying ReLU problem?

A ReLU neuron can become stuck in the negative region. Since $\text{ReLU}(x)=0$ and its derivative is 0 for $x < 0$, the neuron may receive zero gradient and stop updating. This can happen due to large learning rates, poor initialization, or negative biases.

Solutions: Leaky ReLU, PReLU, better initialization, or smaller learning rates.

Q16. Compare ReLU, Leaky ReLU, and PReLU.

ReLU: slope 0 for negative inputs.

Leaky ReLU: uses a fixed small negative slope, reducing the risk of dying neurons.

PReLU: similar to Leaky ReLU, but the negative slope is learned during training.

Leaky ReLU reduces the risk of dying ReLUs; it does not guarantee useful gradients.

Q17. What is the difference between sigmoid/tanh vanishing gradients and dying ReLU?

Sigmoid/tanh can produce very small gradients when saturated, so gradients shrink through repeated multiplication during backpropagation. A dying ReLU has an exactly zero derivative for a neuron stuck in the negative region, so that neuron may stop receiving updates entirely.

5. Output Activation Functions

Q18. What activation is used for regression output?

Typically a linear/identity output: $\hat{y} = z$. This allows unrestricted continuous predictions. It is commonly paired with MSE or MAE. MSE does not mathematically require a linear output, but linear output is the standard choice for unrestricted regression.

Q19. What activation is used for binary classification?

Sigmoid:

$$\sigma(z) = 1 / (1 + e^{-z})$$

It maps the output to (0,1), interpreted as $P(y=1|x)$. It is typically paired with binary cross-entropy.

Q20. What activation is used for mutually exclusive multiclass classification?

Softmax converts logits into a probability distribution over K classes:

$$P(y=k|x) = \exp(z_k) / \sum_j \exp(z_j)$$

The probabilities sum to 1. The predicted class is typically argmax of the probabilities. It is commonly paired with categorical cross-entropy.

Q21. Why not use 10 independent sigmoid neurons for 10 mutually exclusive classes?

Independent sigmoid outputs represent independent binary probabilities and do not have to sum to 1. Multiple classes can simultaneously receive high probabilities. Softmax creates competing class probabilities that sum to 1. Sigmoid outputs are appropriate for multilabel problems where multiple labels can be true simultaneously.

Q22. Multiclass vs multilabel classification?

Multiclass: exactly one of K mutually exclusive classes. Example: dog/cat/lion. Use softmax.

Multilabel: multiple labels can be true simultaneously. Example: an image can contain a dog, person, and car. Use independent sigmoid outputs.

Multitask: multiple related prediction tasks, often with separate heads; each head can use its own suitable activation.

6. Forward Propagation

Q23. What is forward propagation?

Forward propagation is the process of passing an input through the network layer by layer to compute a prediction.

For each layer:

$$Z^{(l)} = W^{(l)}A^{(l-1)} + b^{(l)}$$

$$A^{(l)} = f(Z^{(l)})$$

The final output is then used to compute the loss.

Q24. Example: 3 inputs → 4 hidden neurons → 2 outputs. Write the forward pass.

Using column-vector convention:

$$Z^{(1)} = W^{(1)}X + b^{(1)}, \text{ where } W^{(1)} \text{ is } 4 \times 3$$

$$A^{(1)} = \text{ReLU}(Z^{(1)})$$

$$Z^{(2)} = W^{(2)}A^{(1)} + b^{(2)}, \text{ where } W^{(2)} \text{ is } 2 \times 4$$

If this is 2-class mutually exclusive classification, apply softmax to $Z^{(2)}$.

Q25. What are Z and A?

Z is the **pre-activation**: the weighted sum plus bias.

$$Z = WX + b$$

A is the **activation** after applying the activation function.

$$A = f(Z)$$

Example: if $Z = [-2, 3, -1, 5]$, ReLU gives $A = [0, 3, 0, 5]$.

Q26. What happens with a batch of 32 samples and 3 features?

If samples are rows, X has shape 32×3 . For a $3 \rightarrow 4 \rightarrow 2$ network:

X: 32×3

W_1 : 3×4 , b_1 : 1×4

Z_1/A_1 : 32×4

W_2 : 4×2 , b_2 : 1×2

Output: 32×2

Note: Some mathematical conventions store samples as columns, producing transposed shapes. Always state your convention.

7. Parameter Counting

Q27. How many trainable parameters are in a $3 \rightarrow 4 \rightarrow 2$ network?

First layer: 4×3 weights + 4 biases = 16.

Second layer: 2×4 weights + 2 biases = 10.

Total = 26 trainable parameters.

Q28. General parameter-count formula for a fully connected layer?

For n_{in} input units and n_{out} output units:

$$\begin{aligned} \text{Parameters} &= n_{in} \times n_{out} + n_{out} \\ &= n_{out}(n_{in} + 1) \end{aligned}$$

The extra n_{out} term is one bias per output neuron.

8. Parameters, Activations, and Initialization

Q29. Parameter vs hyperparameter?

Parameters: learned from data, e.g. weights and biases.

Hyperparameters: selected/configured by the user or training setup, e.g. learning rate, number of layers, hidden width, batch size, epochs.

Q30. Why can't all weights be initialized to zero?

The main issue is **symmetry**. If neurons in a layer start with identical weights (and identical biases), they produce identical activations and receive identical gradients. They remain identical and learn the same features, defeating the purpose of multiple neurons.

Key phrase: zero weights $\rightarrow$ symmetry $\rightarrow$ identical gradients $\rightarrow$ identical neurons.

Q31. Can biases be initialized to zero?

Often yes. The symmetry problem is primarily caused by initializing all weights identically. Weights are usually initialized with methods such as Xavier/Glorot or He initialization to break symmetry and maintain reasonable activation/gradient scales.

Q32. Weight vs bias?

Weights determine how strongly input features influence a neuron and, geometrically, affect the orientation of a decision boundary. Bias shifts the activation threshold or position of the boundary.

$$z = \mathbf{w}^T \mathbf{x} + b$$

Q33. Activation vs parameter?

A parameter is learned and stored by the model, such as $\mathbf{W}$ or b . An activation is a value computed dynamically during the forward pass for a particular input.

$$A = f(\mathbf{W}\mathbf{X} + b)$$

9. Rapid-Fire Interview Answers

Q34. Why is bias important in a simple line?

Without bias, $y = wx$ always passes through the origin. Adding b gives $y = wx + b$, allowing the line to shift vertically and better fit the data.

Q35. Why does ReLU help create complex functions?

Each ReLU introduces active/inactive regions and creates piecewise-linear behavior. Combining many neurons and layers creates many linear regions that can approximate complex nonlinear functions.

Q36. Does ReLU itself prevent overfitting like dropout?

No. ReLU creates sparse activations, but it is not primarily a regularization method. Dropout randomly removes activations during training to regularize the model. They are different mechanisms.

Q37. What is the core distinction between representational capacity and learning?

A network may be capable of representing a function, but training still needs to find suitable parameters. The Universal Approximation Theorem concerns representational capacity, not guaranteed optimization or generalization.

Day 1 — Ultra-Quick Revision Sheet

Concept	Remember
Neuron	$z = w^T x + b$; $a = f(z)$
Weight	Controls feature influence; learned parameter
Bias	Shifts activation/decision boundary; learned parameter
Linear-only network	Multiple linear layers collapse into one affine transformation
Activation	Introduces nonlinearity
ReLU	$\max(0, x)$; nonlinear, piecewise linear
Dying ReLU	Negative region $\rightarrow$ derivative 0 $\rightarrow$ neuron may stop updating
Leaky ReLU	Fixed small negative slope
PReLU	Learnable negative slope
Sigmoid	Binary classification; output in (0,1)
Softmax	Mutually exclusive multiclass; probabilities sum to 1
Multilabel	Multiple labels can be true $\rightarrow$ independent sigmoid outputs
Regression output	Usually linear/identity
Forward pass	$Z = WA_{\text{prev}} + b$; $A = f(Z)$
Z vs A	Z = pre-activation; A = post-activation
Parameters	Weights and biases learned by training
Hyperparameters	Learning rate, depth, width, batch size, epochs
Zero weight init	Symmetry problem; identical neurons/gradients
UAT	Sufficiently wide/deep network can approximate broad class of continuous functions under conditions
Deep networks	Hierarchical/compositional representations; often more parameter-efficient
No hidden layer + softmax	Multinomial logistic regression; linear decision boundaries

Day 1 priority traps: (1) ReLU is nonlinear but piecewise linear. (2) UAT concerns representation, not guaranteed training. (3) Zero weight initialization causes symmetry. (4) Input feature count does not determine class count. (5) Multiclass $\neq$ multilabel. (6) ReLU sparsity is not the same as dropout.